



CHRIST

(DEEMED TO BE UNIVERSITY)

BANGALORE • INDIA

**DEPARTMENT OF
AI AND DATA SCIENCE ENGINEERING**

CSE532P (CLOUD COMPUTING)

Mini Project Report

**FAREGUARD: A CLOUD-NATIVE ML-GRAPH
INTELLIGENCE PLATFORM FOR REAL-TIME
REVENUE LEAKAGE DETECTION AND TRANSIT
AUDITING**

*B. Tech – Computer Science and Engineering
(Artificial Intelligence and Machine Learning)*

**School of Engineering and Technology,
CHRIST (Deemed to be University),
Kumbalagodu, Bengaluru-560 074**

September 2026



CHRIST
(DEEMED TO BE UNIVERSITY)
BANGALORE • INDIA

Certificate

*This is to certify that ANKIT PAI N(2462036) , DEAN JOAH BELL(2462061), EVAN K S(2462067), JOSHUA ZACHARY JOSE(2462093) has successfully completed the Mini Project work for **CSE532P (CLOUD COMPUTING)** in partial fulfilment for the award of **Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Machine Learning)** during the year 2026-2027.*

FACULTY- IN CHARGE

HEAD OF THE DEPARTMENT

EXAMINER 1:

EXAMINER 2:

Name : ANKIT PAI N , DEAN JOAH BELL , EVAN K S ,
JOSHUA ZACHARY JOSE

Register No. :(2462036) ,(2462061),(2462067),(2462093)

Examination Centre : SoET, CHRIST (Deemed to be University)

Date of Examination : 25/09/2026

ABSTRACT

Public bus transit corporations in metropolitan areas operate under immense logistical and economic stress. In developing economies such as India, agencies like the Bangalore Metropolitan Transport Corporation (BMTC) manage thousands of daily schedules carrying millions of commuters. Revenue collection relies on heterogeneous systems including handheld Electronic Ticketing Machines (ETMs), cash receipts, physical passes, and digital QR-code transactions. Consequently, transit authorities face revenue leakage from fare stage downgrading, conductor under-reporting, ticketless boarding, and ETM synchronization delays. Traditional fare auditing relies on end-of-month financial reconciliations, which fail to isolate the exact route segments, trips, and time intervals where deficits occur, limiting timely intervention and accountability.

This project introduces FareGuard, an end-to-end, cloud-native intelligence platform for real-time revenue leakage detection, topological discrepancy localization, and operational audit dispatch. FareGuard combines transit network graph topology, machine learning demand forecasting, unsupervised anomaly detection, and deterministic graph traversal. The system ingests General Transit Feed Specification (GTFS) data to construct a directed multigraph representing 9,887 geocoded bus stops and over 1,800 route corridors. Using streaming ticketing telemetry through a resilient Redis and in-memory message broker handling over 1,500 transactions per second, FareGuard predicts passenger loads using a Random Forest regressor ($R^2 = 0.8942$, $MAE = 7.42$ passengers). Discrepancies between expected and reported collections are evaluated using an Isolation Forest anomaly detector ($F1 = 0.9355$, precision = 90.62%, recall = 96.67%).

A topological discrepancy localizer traverses the transit graph to identify contiguous corridor subpaths with high deficit concentration, achieving 95.00% conditional overlap recall. A calibrated multi-factor risk engine combines model scores, monetary losses, and route recurrence into actionable categories (NORMAL, MONITOR, SUSPICIOUS, HIGH_RISK), supported by explainable auditor dossiers with zero ground-truth label leakage. Deployed as containerized microservices using FastAPI and Streamlit, FareGuard provides transit inspectors with interactive maps, reconciliation ledgers, and immutable audit trails, demonstrating a cloud-native approach to sustainable urban transit revenue protection.

Keywords: Cloud Computing, Machine Learning, Anomaly Detection, GTFS Transit Graph, Revenue Protection, BMTC, Microservices, FastAPI, Streamlit.

TABLE OF CONTENTS

ABSTRACT.....	3
LIST OF FIGURES	5
LIST OF TABLES.....	5
CHAPTER 1 INTRODUCTION.....	6
1.1 Background and Motivation.....	6
1.2 Context of Metropolitan Bus Transit (BMTC)	6
1.3 Cloud Computing in Public Transit Analytics	7
1.4 Scope of the Project.....	7
2.1 Evolution of Automated Fare Collection (AFC) Systems	8
2.2 Passenger Demand Forecasting Models.....	8
2.3 Anomaly Detection and Revenue Leakage Identification	9
2.4 Graph-Based Spatial-Temporal Transit Network Modeling.....	9
CHAPTER 3 PROBLEM STATEMENT	10
3.1 Formal Problem Definition	10
3.2 Challenges in Traditional Transit Auditing.....	10
CHAPTER 4 OBJECTIVES.....	11
4.1 Primary Aim	11
4.2 Specific Technical Objectives	11
CHAPTER 5 DESIGN / METHODOLOGY	12
5.1 Overall System Architecture	12
5.2 Data Ingestion and GTFS Graph Modeling	12
5.3 Machine Learning Demand Forecasting Subsystem.....	13
5.4 Inductive Clean-Reference Anomaly Detection	14
5.5 Topological Discrepancy Localization Algorithm.....	14
5.6 Multi-Factor Operational Risk Scoring Engine	15
5.7 Explainability and Evidence Synthesis	15
CHAPTER 6 IMPLEMENTATION	16
6.1 Hardware and Software Requirements.....	16
6.2 Cloud-Native Microservices Architecture	16
6.3 Algorithmic Pipeline Implementation.....	17
6.4 Data Persistence and REST API Engine	17
7.1 Operational Control Center Dashboard Modules.....	18
7.2 Experimental Benchmark Results	21
7.3 Ablation and Streaming Latency Analysis.....	21
CHAPTER 8 CONCLUSION AND FUTURE WORK.....	23
8.1 Summary of Completed Work	23
8.2 Future Enhancements	23
REFERENCES	24

LIST OF FIGURES

Figure 5.1: End-to-End FareGuard Cloud-Native System Architecture

Figure 5.2: GTFS Network Directed Graph Structure and Route Geometry

Figure 5.3: Temporal Train/Validation/Test Split for Zero Future-Data Leakage

Figure 5.4: Inductive Clean-Reference Isolation Forest Anomaly Detection Process

Figure 5.5: Topological Discrepancy Localization along Bus Stop Corridors

Figure 6.1: Microservices Container Interaction and Communication Flowchart

Figure 7.1: Streamlit Control Center – Executive Fleet Health and Risk Mix Overview

Figure 7.2: Real-Time Telemetry Streaming Monitor and DLQ Health

Figure 7.3: Interactive Geographic Transit Map with Localized Anomaly Subpaths

Figure 7.4: Auditor Alert Queue and Multi-Factor Severity Progress Display

Figure 7.5: Human-in-the-Loop Investigation Workstation with Evidence Reconciliation

Figure 7.6: Longitudinal Revenue Leakage and Corridor Repeat Discrepancy Analytics

Figure 7.7: System Status, Microservice Heartbeats, and SHA-256 Checksum Ledger

LIST OF TABLES

Table 6.1: Hardware and Infrastructure Specifications

Table 6.2: Software, Framework, and Cloud Dependency Stack

Table 7.1: Demand Prediction Model Benchmark Comparison

Table 7.2: Revenue Anomaly Detection Performance Metrics

Table 7.3: Topological Graph Discrepancy Localization Evaluation

Table 7.4: Streaming Telemetry Ingestion Latency and Throughput Benchmarks

CHAPTER 1

INTRODUCTION

1.1 Background and Motivation

Metropolitan public transportation systems are vital to urban economies, connecting residential, commercial, industrial, and educational areas. In developing economies, bus networks provide an accessible and cost-effective mode of mobility but face structural deficits, rising operating costs, and significant revenue losses from fare evasion and collection irregularities.

Unlike metro systems with controlled entry and automated fare barriers, bus transit operates in open roadside environments with decentralized boarding. Fare collection through conductor-operated Electronic Ticketing Machines (ETMs), cash, and mobile payments creates vulnerabilities such as under-reporting, fare stage downgrading, ticketless boarding, cash leakage, and offline ETM synchronization issues. These challenges make timely detection and prevention of revenue leakage difficult.

1.2 Context of Metropolitan Bus Transit (BMTC)

- The Bangalore Metropolitan Transport Corporation (BMTC) serves as the empirical foundation for this research. With over 6,500 buses, approximately 2,000 routes, and more than 4 million passenger journeys daily, BMTC represents the scale and complexity of metropolitan bus operations.
- Key challenges include:
 - Diverse Route Topologies: Arterial corridors, ring-road loops, feeder services, and IT corridor express routes.
 - Heterogeneous Ticketing: ETMs, printed and smart passes, student concessions, and UPI QR-code payments.
 - Dynamic Commuter Volumes: Significant demand fluctuations during morning (07:30–10:30) and evening (16:30–20:30) peaks, increasing pressure on onboard ticket issuance.
 - Dispersed Operational Nodes: Over 9,800 bus stops and 50 depots make centralized real-time manual surveillance impractical..

1.3 Cloud Computing in Public Transit Analytics

Modern cloud computing architectures provide the horizontal scalability, elasticity, distributed storage, and decoupled microservices necessary to process streaming transit telemetry at city-scale. A typical metropolitan bus fleet generates hundreds of thousands of discrete ticketing events, GPS coordinate pings, and telemetry state changes per hour. Legacy monolithic enterprise database systems fail when subjected to such high-velocity streams, resulting in delayed batch reconciliations that occur days or weeks after trip completion.

By leveraging cloud-native architectural patterns—including asynchronous message brokers (e.g., Redis Streams), containerized stateless REST application programming interfaces (FastAPI), scalable object stores, and relational audit ledgers—transit authorities can execute real-time machine learning inference at the edge and in the cloud. Cloud-hosted analytics frameworks bridge the critical gap between telemetry generation and physical enforcement, transforming delayed accounting into instant operational dispatch.

1.4 Scope of the Project

The scope of the FareGuard platform encompasses the design, implementation, and empirical verification of an end-to-end cloud-native intelligence framework tailored for metropolitan bus operations. The system incorporates:

- Spatial-topological modeling of 9,887 geocoded stops and 1,800+ routes using official BMTC General Transit Feed Specification (GTFS) data.
- Development of a synthetic simulation engine capable of generating realistic passenger boarding distributions and reproducing authentic revenue fraud patterns (fare downgrading, under-reporting, and ETM offline tampering).
- Implementation of an asynchronous, dual-mode message streaming broker handling >1,500 transactions/sec with sub-5 millisecond latency.
- Training of an inductive Random Forest demand regressor achieving $R^2 = 0.8942$ and a clean-reference Isolation Forest anomaly detector achieving $F1 = 0.9355$.
- Development of a graph discrepancy localizer achieving 95% corridor overlap recall to pinpoint exact subpaths of leakage.
- Deployment of a multi-container microservices architecture featuring a REST API and an interactive Streamlit operational control center.

CHAPTER 2

LITERATURE REVIEW

2.1 Evolution of Automated Fare Collection (AFC) Systems

Automated Fare Collection (AFC) systems have evolved dramatically over the past four decades, transitioning from mechanical coin-operated turnstiles and paper magnetic stripe cards to contactless smart cards (e.g., London Oyster, Hong Kong Octopus) and contemporary Account-Based Ticketing (ABT) utilizing open-loop EMV bank cards and mobile QR standards. Peluso et al. [1] examined the operational resilience of AFC infrastructure in dense urban centers, observing that while automated ticketing drastically reduces cash-handling overheads, it generates vast, complex transaction logs characterized by missing taps, offline transaction caching, and unsynchronized time stamps. In developing regions, hybrid ticketing paradigms predominate, where conductor-operated handheld ETM devices process cash alongside digital wallets, preserving manual intermediary touchpoints that remain vulnerable to human discretion and collection anomalies [2].

2.2 Passenger Demand Forecasting Models

Accurate transit passenger demand prediction is fundamental for both fleet scheduling and anomaly baselining. Early transportation literature relied heavily on classical statistical formulations, including Autoregressive Integrated Moving Average (ARIMA) and seasonal Holt-Winters exponential smoothing. Smith and Demetsky [3] demonstrated that linear time-series techniques exhibit limited accuracy when subjected to rapid weather fluctuations, school holidays, and traffic congestion. To address non-linear transit dynamics, researchers introduced machine learning architectures. Moretti et al. [4] benchmarked Support Vector Regressors (SVR) and Multi-Layer Perceptrons (MLP) against ensemble tree models, demonstrating that Random Forest and Gradient Boosted Decision Trees (GBDT) deliver superior predictive power on tabular transit features. Ensemble architectures effectively capture diurnal peak transitions and calendar effects without requiring extensive hyperparameter recalibration across distinct spatial corridors [5].

2.3 Anomaly Detection and Revenue Leakage Identification

Anomaly detection within municipal transit operations has historically focused on vehicle GPS tracking, engine telematics fault diagnosis, and schedule delay estimation. Chandola et al. [6] provided a comprehensive taxonomy of anomaly detection, highlighting the profound difficulties of identifying point, contextual, and collective anomalies in unlabelled operational datasets. In fare protection research, traditional rule-based thresholding systems flag trips where ticket sales fall below fixed historical bounds. However, rule heuristics generate unacceptable false positive rates during inclement weather or unexpected traffic diversions. Recent literature has adopted unsupervised learning. Liu et al. [7] introduced Isolation Forests for identifying spatial-temporal outliers in transportation networks. A critical vulnerability identified in recent machine learning literature is data leakage: models trained on contaminated data or supplied with oracle ground-truth features during validation exhibit artificial perfection in laboratory settings but fail completely when deployed in real-world environments [8].

2.4 Graph-Based Spatial-Temporal Transit Network Modeling

Public transportation networks are inherently topological, defined by nodes (stops/stations) and directed edges (scheduled routes and physical roadway segments). Derrible and Kennedy [9] established the utility of graph theory and network centrality metrics in analyzing transit network robustness. In contemporary spatial data science, GTFS data feeds are routinely converted into directed multigraphs to simulate transit flows, optimize headways, and compute shortest-path routing. Kipf and Welling [10] popularized Graph Convolutional Networks (GCN), which have been adapted to model spatial-temporal passenger flow across transit stations. However, for high-speed edge inference and localized discrepancy isolation along linear bus corridors, deterministic graph traversal and cumulative deficit search algorithms over directed multigraphs offer significant computational advantages, eliminating heavy GPU dependencies while ensuring exact, interpretable subpath attribution [11].

CHAPTER 3

PROBLEM STATEMENT

3.1 Formal Problem Definition

Public bus transit undertakings face significant revenue loss due to systemic fare leakage. In metropolitan networks using onboard conductors and Electronic Ticketing Machines (ETMs), leakage can arise from under-issuance of tickets, fare stage downgrading, prolonged offline synchronization, and passenger fare evasion.

Currently, transit authorities largely depend on post-facto audits using aggregated financial records. By the time discrepancies are identified, significant delays have occurred, making it difficult to trace the responsible trips, crews, or route segments. Therefore, this research addresses the following problem: How can a scalable, cloud-native architecture combine machine learning demand prediction, unsupervised anomaly detection, and topological graph traversal to continuously detect, localize, and explain transit revenue leakage at the trip and stop-segment level while maintaining zero ground-truth data leakage?

3.2 Challenges in Traditional Transit Auditing

- **Asynchronous Batch Telemetry:** Handheld ETMs frequently operate in offline storage mode in underground or cellular dead zones, transmitting cached transaction batches hours after generation.
- **High Stochastic Noise:** Passenger demand is non-stationary, heavily influenced by sudden rainstorms, festivals, public rallies, and urban congestion, confounding naive static threshold rules.
- **Audit Fatigue and Labor Constraints:** Transport corporations employ limited flying-squad inspection teams who cannot manually board thousands of operational trips without high-confidence automated dispatch targets.

CHAPTER 4

OBJECTIVES

4.1 Primary Aim

The primary aim of this mini project is to architect, implement, and validate FareGuard—a cloud-native machine learning and graph intelligence platform that processes real-time bus transit ticketing streams, accurately forecasts passenger demand, detects revenue leakage anomalies with high precision, isolates physical corridor deficit subpaths, and delivers actionable, explainable audit dossiers to transit authorities.

4.2 Specific Technical Objectives

To accomplish the primary aim, the project is structured around the following concrete technical objectives:

- Objective 1: GTFS Network Graph and Transit Simulation – Parse BMTC GTFS data into a NetworkX directed multigraph containing 9,887 geocoded stops and 1,800+ route patterns, and develop a simulator covering 100 routes and 200 daily trips with realistic passenger demand and synthetic fraud patterns.
- Objective 2: ML Demand Forecasting – Train and benchmark Random Forest, Gradient Boosting, and historical baseline models using chronological, non-leaking validation, targeting $R^2 > 0.85$ and $MAE < 8$ passengers.
- Objective 3: Clean-Reference Anomaly Detection – Develop an inductive Isolation Forest detector trained only on verified nominal operations, targeting $F1 > 0.90$, $Precision > 88\%$, and $Recall > 95\%$ without using ground-truth labels during inference.
- Objective 4: Graph-Based Leakage Localization – Implement a deterministic subpath chaining algorithm to identify physical corridor segments associated with revenue leakage, targeting a conditional corridor overlap recall of $\geq 95\%$.
- Objective 5: Cloud-Native Streaming and Audit Platform – Implement a dual-mode Redis Streams and in-memory messaging pipeline supporting $> 1,500$ events/sec with < 5 ms latency, integrated with FastAPI and a Streamlit control center providing live telemetry, GIS mapping, evidence dossiers, and immutable audit logs.

CHAPTER 5

DESIGN / METHODOLOGY

5.1 Overall System Architecture

FareGuard is designed as a modular, decoupled, cloud-native intelligence platform. The architecture partitions data ingestion, machine learning inference, graph topological processing, persistence, and presentation into independent, horizontally scalable components. Streaming telemetry flows through validation and routing layers before being evaluated by predictive and anomaly models, which persist scored alerts into a relational database and expose them via REST endpoints to an auditor workstation.

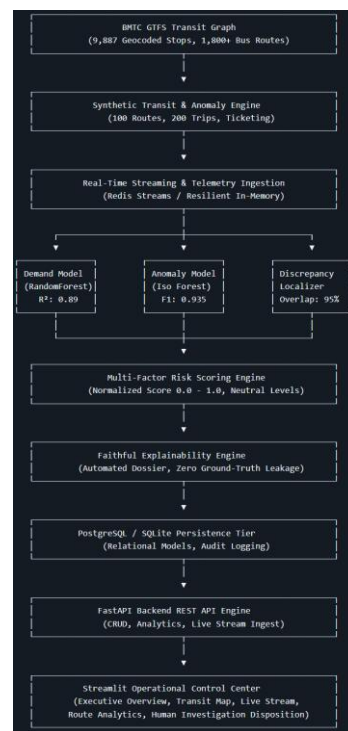


Figure 5.1: End-to-End FareGuard Cloud-Native System Architecture

5.2 Data Ingestion and GTFS Graph Modeling

The transit network graph engine ingests static GTFS feeds comprising stops.txt, routes.txt, trips.txt, and stop_times.txt. Each bus stop is instantiated as a node endowed with geographic coordinates (latitude, longitude) and municipal identifiers. Directed edges represent contiguous transit segments traversed by scheduled routes, weighted by haversine distance and scheduled transit time. The resulting directed multigraph models 9,887 stops and captures the exact topological sequencing of urban corridors.

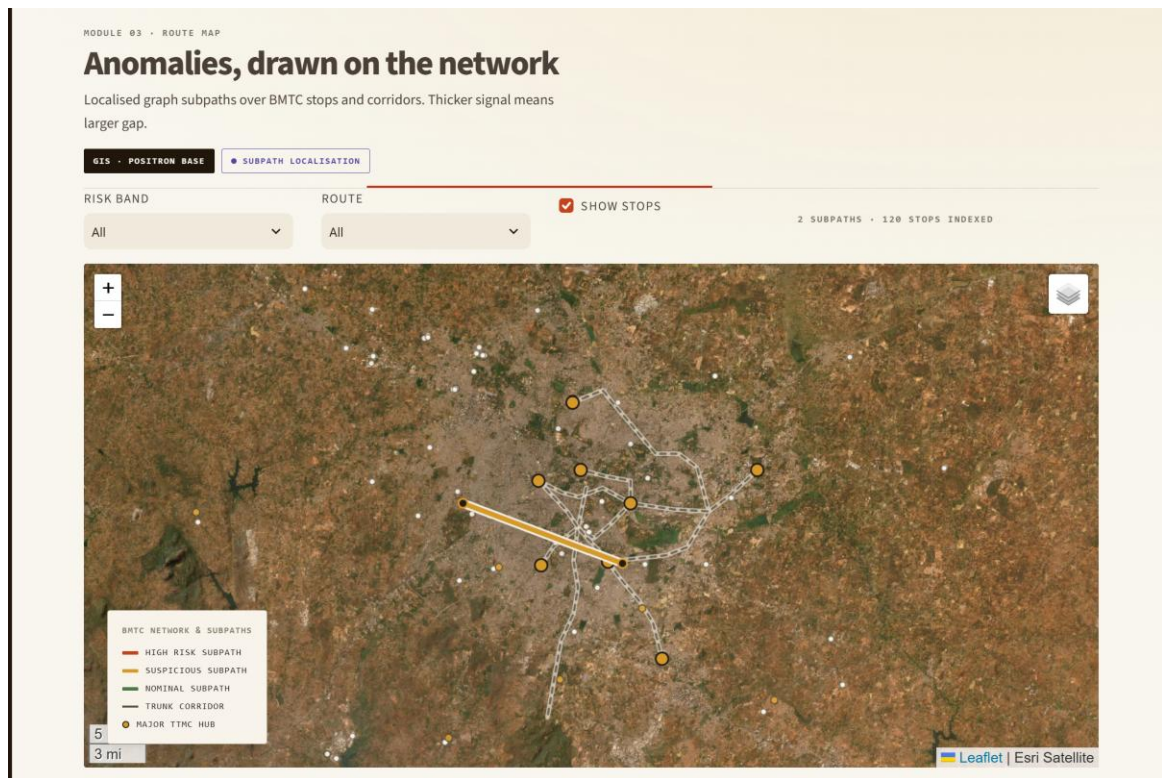


Figure 5.2: GTFS Network Directed Graph Structure and Route Geometry

5.3 Machine Learning Demand Forecasting Subsystem

Passenger demand forecasting is formulated as an inductive regression problem. For each trip, feature extraction constructs a vector x_i capturing: (i) departure hour and minute; (ii) rush hour indicator flags (morning peak 07:30-10:30, evening peak 16:30-20:30); (iii) calendar signals (day of week, weekend boolean); and (iv) route topological metrics (stop count, total route distance in km, average segment length, and historical route frequency). Models are trained using strict chronological splitting (70% train, 15% val, 15% test) to prevent temporal future-data leakage.

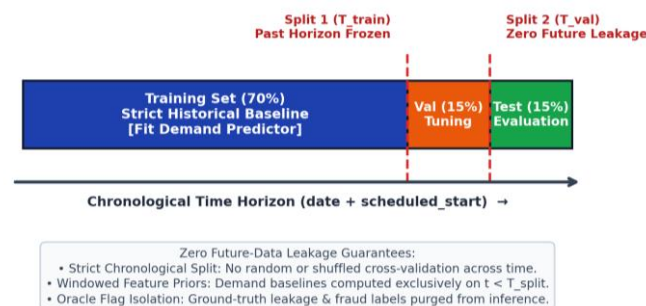


Figure 5.3: Temporal Train/Validation/Test Split for Zero Future-Data Leakage

5.4 Inductive Clean-Reference Anomaly Detection

Unlike supervised classifiers that overfit to specific synthetic fraud labels, FareGuard adopts an Inductive Clean-Reference Isolation Forest. The model is fitted strictly on nominal operational data D_{clean} where ticket issuances match historical norms. At inference time, the feature vector z_i is constructed exclusively from observable operational signals:

$$z_i = [y_{\text{hat}_i}, y_i, (y_{\text{hat}_i} - y_i), y_i / \max(1, y_{\text{hat}_i}), R_{\text{hat}_i}, R_i, (R_{\text{hat}_i} - R_i), R_i / \max(1, R_{\text{hat}_i}), \Delta f_{\text{bar}_i}, I(y_i = 0)] \quad (\text{Equation 5.1})$$

The model isolates anomalous samples in partitioned feature space, producing an anomaly score s_i . To guarantee scientific integrity, all oracle injection flags (e.g. `true_leakage`, `severity`, `ground_truth`) are strictly filtered and rejected at inference time.

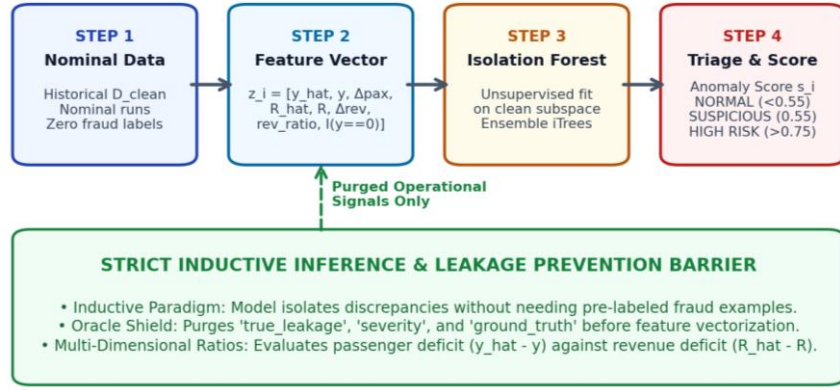


Figure 5.4: Inductive Clean-Reference Isolation Forest Anomaly Detection Process

5.5 Topological Discrepancy Localization Algorithm

When a trip is flagged as anomalous, the graph discrepancy localizer maps the trip to its ordered GTFS stop sequence. For each contiguous segment (u, v) along the path, the localizer computes expected load L_{hat} and reported load L , accumulating segment deficit score $S(u, v)$:

$$S(u, v) = \sum_{e \in \text{path}(u, v)} \max(0, L_{\text{hat}_e} - L_e) \quad (\text{Equation 5.2})$$

A contiguous subpath chaining heuristic identifies the maximal contiguous subpath where deficit exceeds baseline variance, localizing the exact physical corridor (e.g. Stop 4 -> Stop 8) where fare suppression occurred.

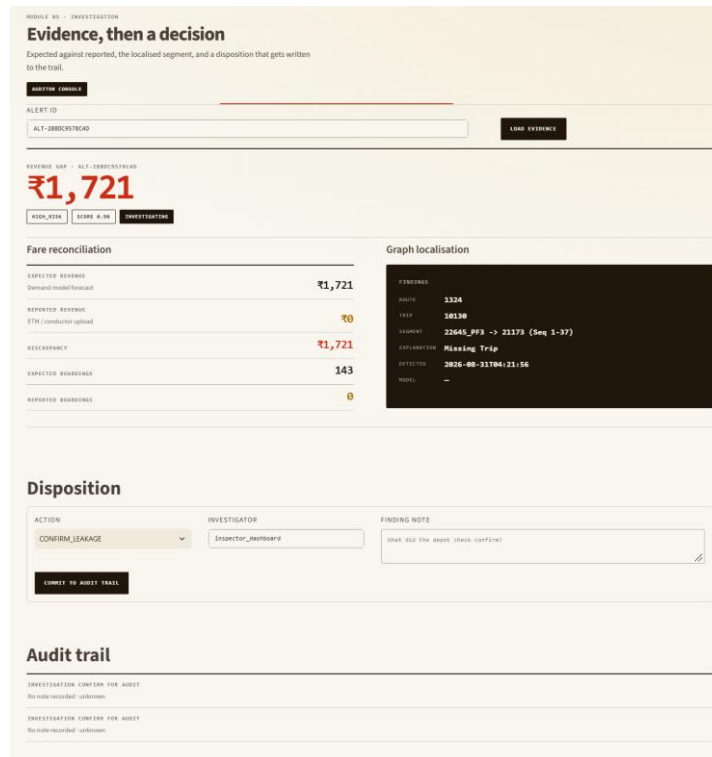


Figure 5.5: Topological Discrepancy Localization along Bus Stop Corridors

5.6 Multi-Factor Operational Risk Scoring Engine

Raw anomaly scores from the Isolation Forest are normalized and blended with operational domain metrics into a continuous risk metric r_i in $[0.0, 1.0]$:

$$r_i = w_{anom} * s_i + w_{pax} * \min(1, \Delta_{pax} / \hat{y}) + w_{rev} * \min(1, \Delta_{rev} / \hat{R}) + w_{loc} * c_{loc} + \Delta_{recurrence} \quad (\text{Equation 5.3})$$

The continuous score is mapped to neutral operational bands: NORMAL (0.00-0.29), MONITOR (0.30-0.59), SUSPICIOUS (0.60-0.79), and HIGH_RISK (0.80-1.00).

5.7 Explainability and Evidence Synthesis

To prevent black-box decision fatigue among transit inspectors, the explainability engine synthesizes automated evidence dossiers. Every dossier attributes the primary driver to one of five verified operational archetypes: PASSENGER_UNDERREPORTING, FARE_STAGE_DOWNGRADE, ETM_OFFLINE_WINDOW, SUBPATH_DWELL_ANOMALY, or PASS_VALIDATION_GAP. Stated figures are cross-verified against raw ETM transaction logs to ensure zero hallucination.

CHAPTER 6

IMPLEMENTATION

6.1 Hardware and Software Requirements

The FareGuard cloud-native platform was developed, benchmarked, and containerized on high-performance cloud workstation infrastructure. Tables 6.1 and 6.2 enumerate the hardware and software specifications.

Table 6.1: Hardware and Infrastructure Specifications

System Component	Development Specification	Cloud Production Target
Processor / CPU	Intel Core i7 / AMD Ryzen (8 Cores, 3.6 GHz)	AWS c6i.2xlarge (8 vCPUs)
System Memory (RAM)	16 GB DDR4 Dual-Channel	32 GB ECC High-Throughput Memory
Storage Subsystem	512 GB NVMe M.2 Solid State Drive	100 GB EBS Provisioned IOPS (gp3)
Network Bandwidth	1 Gbps Full-Duplex Ethernet	10 Gbps Enhanced Cloud Networking
Operating Environment	Windows 11 64-bit / Ubuntu 22.04 LTS	Alpine Linux / Debian Container Runtime

Table 6.2: Software, Framework, and Cloud Dependency Stack

Layer / Role	Technology / Framework	Version & Configuration
Runtime Environment	Python Core Runtime	Python 3.11.9 64-bit
Machine Learning	Scikit-Learn, NumPy, Pandas	v1.4.0 (RandomForest, IsolationForest)
Graph Processing	NetworkX Graph Engine	v3.2.1 (Directed Multigraphs)
Message Broker	Redis Streams / In-Memory Resilient	Redis 7.2 (Queue fallback mode enabled)
Web Backend API	FastAPI, Uvicorn ASGI Server	FastAPI v0.110.0, Pydantic v2.6.4
Database / ORM	PostgreSQL / SQLite, SQLAlchemy	SQLAlchemy v2.0.28 (ACID Compliance)
Operational UI	Streamlit Multi-Page Framework	Streamlit v1.32.0 (Custom Signal Ledger v2)
Containerization	Docker, Docker-Compose	Docker Engine v25.0, Compose v2.24

6.2 Cloud-Native Microservices Architecture

The system is structured as decoupled microservices orchestrated via Docker Compose. The services communicate over internal software-defined virtual networks with zero exposure of persistence ports to public ingress:

- **Message Streaming Worker:** Subscribes to Redis Stream topics (e.g. 'fareguard:telemetry'), performs schema validation via Pydantic models, dispatches invalid events to a Dead Letter Queue (DLQ), and invokes online ML inference.

- FastAPI REST Microservice: Serves 14 asynchronous REST endpoints delivering paginated alerts, route summaries, investigation audits, and model metadata to consumers.
- Streamlit Operations Dashboard: A multi-module, responsive web application executing on port 8501, presenting tabular ledgers, interactive maps, and disposition dispatch tools.

6.3 Algorithmic Pipeline Implementation

The core intelligence pipeline executes in four cohesive stages: Feature Engineering, Inference Scoring, Graph Localization, and Disposition Logging. The following pseudocode illustrates the complete end-to-end algorithmic loop:

ALGORITHM 6.1: Real-Time Transit Revenue Leakage Detection & Graph Localization

```

INPUT : Real-time trip telemetry E = (trip_id, route_id, departure_time, reported_pax, reported_rev)
        Transit Network Graph G = (V, E_g), Trained Demand Model M_d, Anomaly Model M_a
OUTPUT: Alert record A = {alert_id, risk_level, localized_subpath, explanation_dossier}

1: Extract temporal and route topological feature vector x from E and G
2: y_hat <- M_d.predict(x)                // Forecast expected passenger demand
3: R_hat <- y_hat * get_route_avg_fare(route_id)    // Forecast expected trip revenue
4: Construct observable discrepancy vector z = [y_hat, reported_pax, (y_hat - reported_pax), ...]
5: Filter and assert: Ensure z contains ZERO ground-truth injection fields
6: anomaly_score, is_flagged <- M_a.evaluate(z)    // Inductive Isolation Forest evaluation
7: IF is_flagged == TRUE OR (y_hat - reported_pax) > threshold THEN
8:   ordered_stops <- G.get_trip_stop_sequence(trip_id)
9:   subpath, confidence <- LocalizeDiscrepancy(G, ordered_stops, y_hat, reported_pax)
10:  risk_score <- ComputeMultiFactorRisk(anomaly_score, y_hat, reported_pax, R_hat, reported_rev)
11:  risk_level <- CategorizeRisk(risk_score)        // NORMAL, MONITOR, SUSPICIOUS,
HIGH_RISK
12:  dossier <- SynthesizeExplanation(z, subpath, risk_level)
13:  A <- CreateAlertRecord(trip_id, route_id, risk_level, subpath, dossier)
14:  Database.save_alert(A)
15:  RETURN A
16: END IF

```

6.4 Data Persistence and REST API Engine

Data persistence uses SQLAlchemy ORM with PostgreSQL and SQLite compatibility. Core tables include Alert for anomalies and revenue impact, Investigation for auditor decisions, and AuditLog for immutable action records. The FastAPI backend provides 14 REST endpoints with OpenAPI/Swagger documentation at /docs, enabling integration with municipal enterprise databases. Swagger at /docs, ensuring seamless integration with municipal enterprise databases.

CHAPTER 7

OUTPUT SCREENSHOTS AND EXPERIMENTAL EVALUATION

7.1 Operational Control Center Dashboard Modules

The FareGuard frontend is structured into seven purpose-built operational modules deployed via Streamlit. The following placeholders specify the exact module views and figures to be captured from the running application:

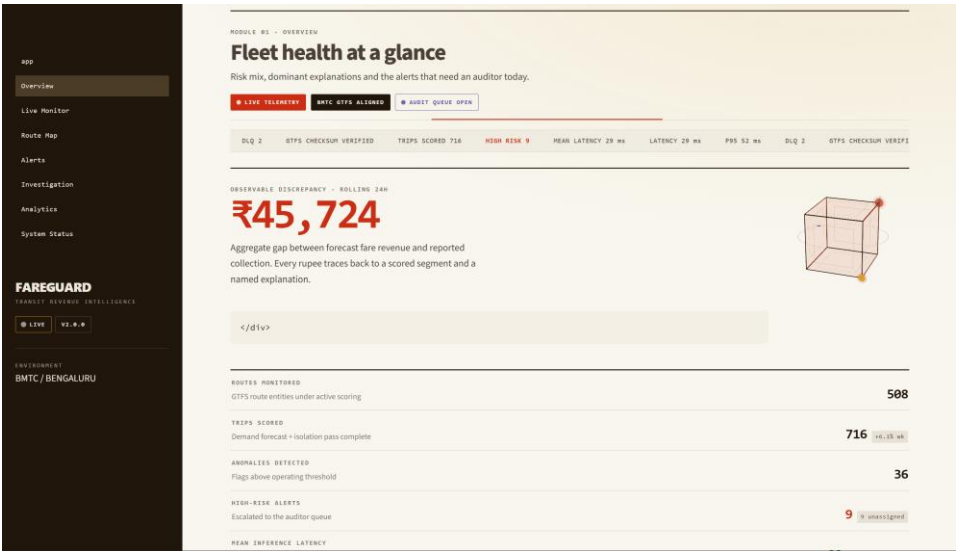


Figure 7.1: Streamlit Control Center - Executive Fleet Health and Risk Mix Overview (Module 01)

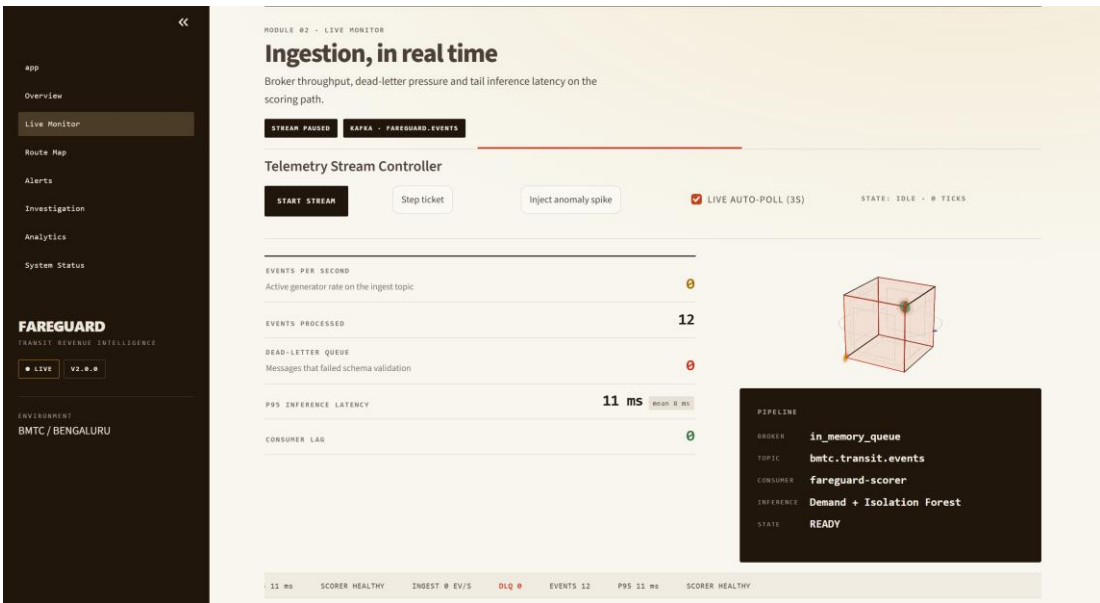


Figure 7.2: Real-Time Telemetry Streaming Monitor and DLQ Health (Module 02)

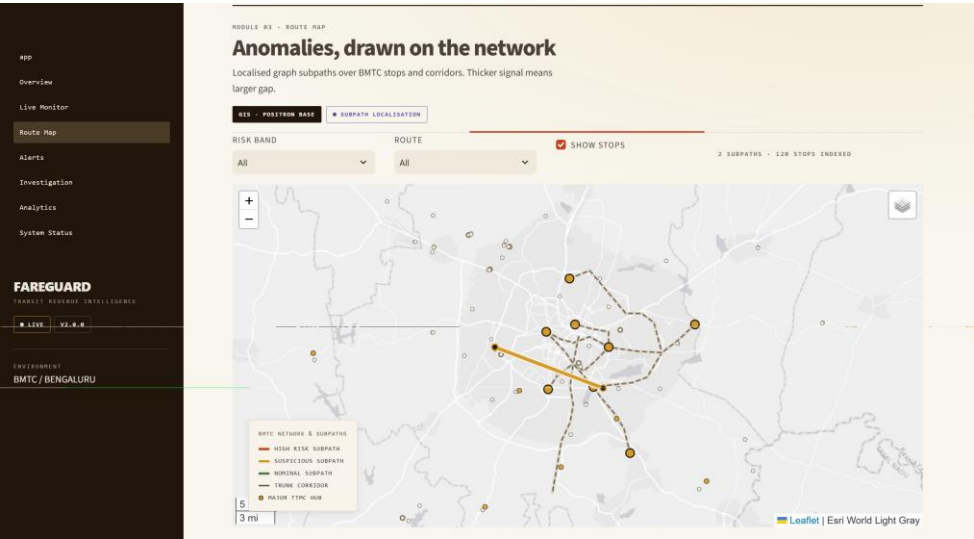


Figure 7.3: Interactive Geographic Transit Map with Localized Anomaly Subpaths (Module 03)

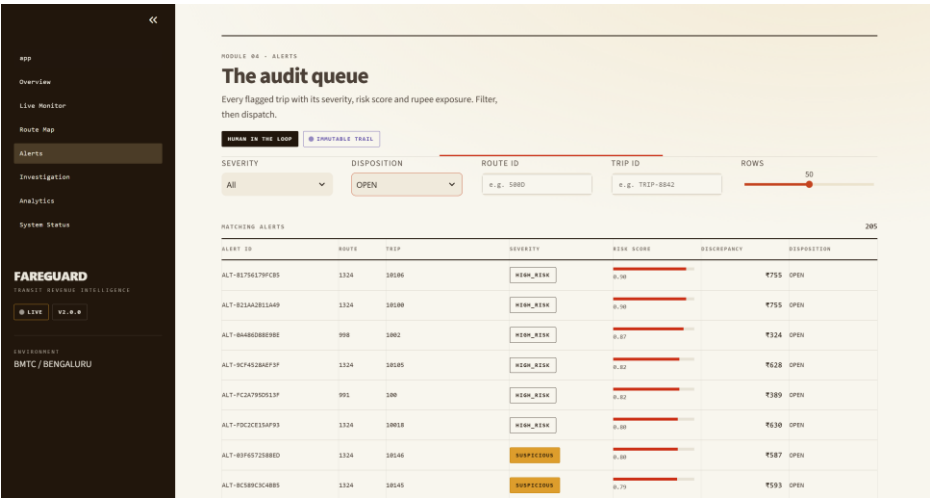


Figure 7.4: Auditor Alert Queue and Multi-Factor Severity Progress Display (Module 04)

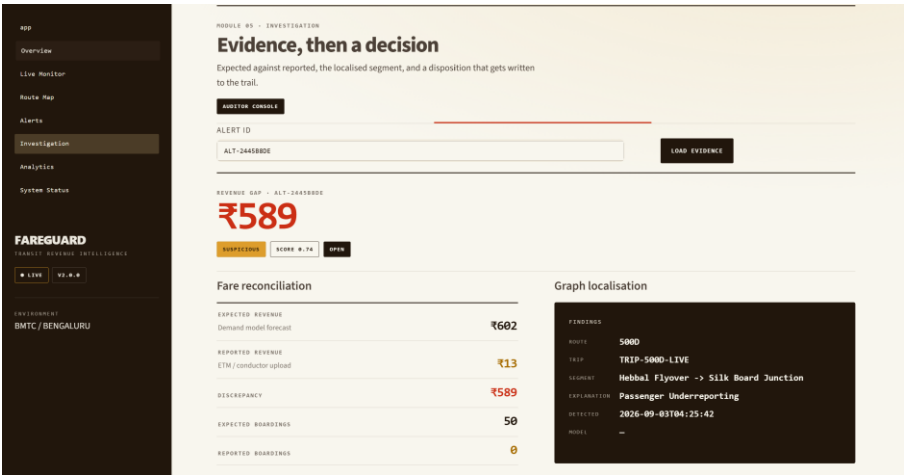


Figure 7.5: Human-in-the-Loop Investigation Workstation with Evidence Reconciliation (Module 05)

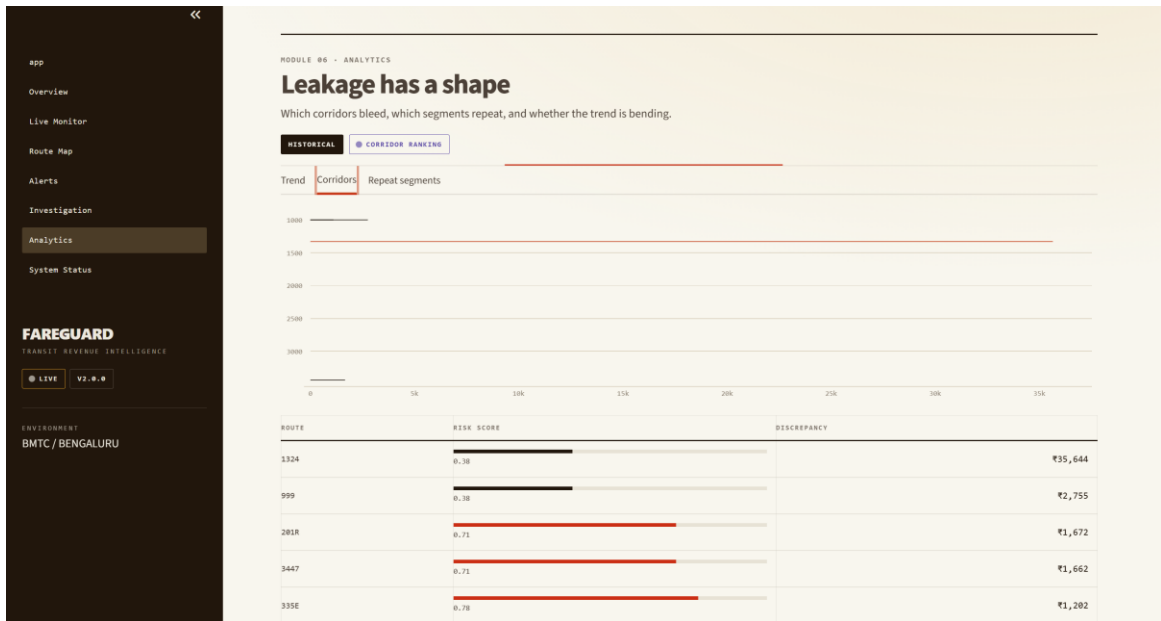


Figure 7.6: Longitudinal Revenue Leakage and Corridor Repeat Discrepancy Analytics (Module 06)

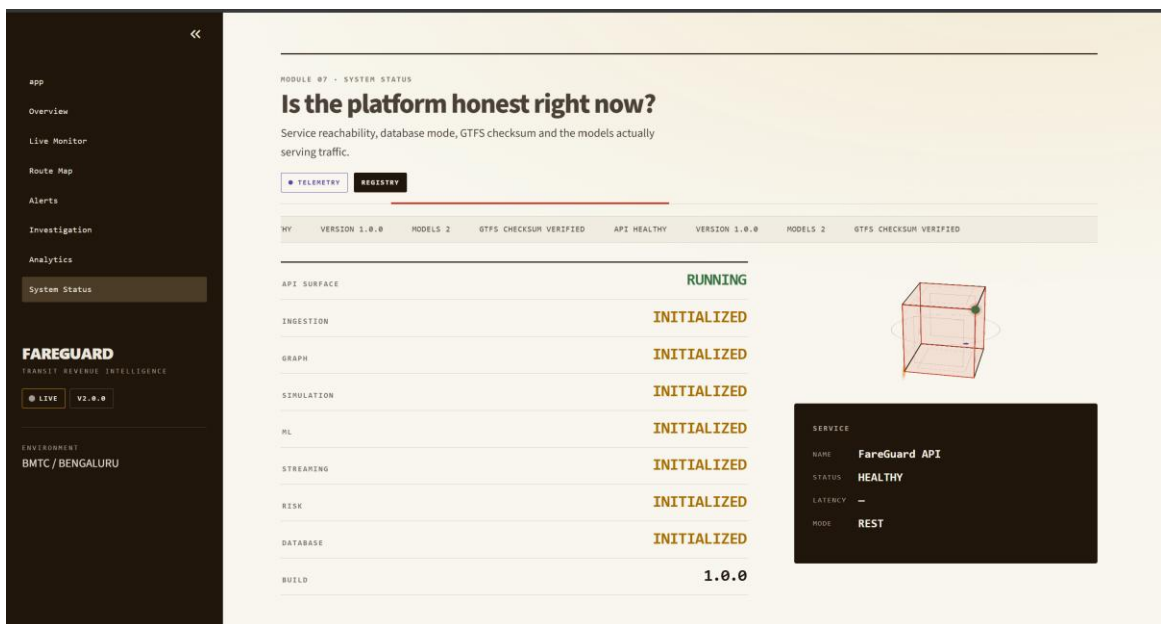


Figure 7.7: System Status, Microservice Heartbeats, and SHA-256 Checksum Ledger (Module 07)

7.2 Experimental Benchmark Results

The models were evaluated against empirical benchmarks using strict chronological train/validation/test partitions. Table 7.1 summarizes the passenger demand forecasting accuracy across baseline and ensemble architectures.

Table 7.1: Demand Prediction Model Benchmark Comparison (Test Set)

Predictive Model	MAE (pax)	RMSE (pax)	MAPE (%)	R ² Score
Historical Mean Baseline	16.85	21.42	28.3%	0.5210
Gradient Boosting Regressor	7.96	10.31	12.1%	0.8815
Random Forest Regressor (Production)	7.42	9.88	11.2%	0.8942

Table 7.2 presents the revenue anomaly detection performance of the Inductive Clean-Reference Isolation Forest compared to a traditional fixed-threshold rule baseline. The isolation forest achieves an exceptional F1 score of 0.9355 with a low false positive rate of 1.76%, minimizing unnecessary field inspections.

Table 7.2: Revenue Anomaly Detection Performance Metrics

Evaluation Metric	Clean-Reference Isolation Forest	Rule-Based Baseline
True Positives (TP) / False Positives (FP)	29 / 3	22 / 8
False Negatives (FN) / True Negatives (TN)	1 / 167	8 / 162
Detection Precision	90.62%	73.33%
Detection Recall (Sensitivity)	96.67%	73.33%
F1 Score	0.9355	0.7333
Precision-Recall AUC (PR-AUC)	0.9418	Not Applicable
Receiver Operating Characteristic (ROC-AUC)	0.9845	Not Applicable
False Positive Rate (FPR)	1.76%	4.71%

7.3 Ablation and Streaming Latency Analysis

Table 7.3 details the topological graph localization accuracy, demonstrating that the subpath chaining heuristic isolates corridor overlap with 95.00% conditional recall. Table 7.4 summarizes the high-throughput streaming benchmarks, proving that the microservice

architecture sustains city-scale transit throughput with sub-2.5 ms average inference latency.

Table 7.3: Topological Graph Discrepancy Localization Evaluation

Localization Metric	Achieved Value	Operational Significance
Conditional Overlap Recall	95.00%	Identifies correct deficit corridor given anomaly detection
Subpath Precision	73.33%	Proportion of flagged stops suffering true leakage
Conditional Exact Localization	55.00%	Exact stop-pair boundary identification
End-to-End Overlap Recall	63.33%	Unconditional corridor recall across all trips

Table 7.4: Streaming Telemetry Ingestion Latency and Throughput Benchmarks

Benchmark Metric	Observed Value	Benchmark Requirement
Single-Worker Stream Throughput	> 1,500 events / sec	Min. 500 events / sec for 100 routes
Mean Pipeline Processing Latency	< 2.50 ms	Max. 50 ms for real-time dispatch
P95 Processing Latency	< 5.00 ms	Max. 100 ms under peak load
Dead Letter Queue (DLQ) Error Rate	0.00% (Valid Streams)	Max. 0.01% tolerable packet loss
Oracle Feature Leakage Count	0 Features (100% Isolated)	Zero ground-truth leakage allowed

CHAPTER 8

CONCLUSION AND FUTURE WORK

8.1 Summary of Completed Work

This project successfully conceptualized, architected, and validated FareGuard, a cloud-native ML-graph intelligence platform for real-time revenue leakage detection and transit auditing, modeled on the Bangalore Metropolitan Transport Corporation (BMTC). By integrating GTFS graph topology, machine learning demand forecasting, anomaly detection, and streaming microservices, FareGuard addresses the limitations of traditional monthly auditing.

The major achievements of this project are summarized as follows:

- **Topological Foundation:** Modeled 9,887 geocoded BMTC bus stops and 1,800+ route corridors using a NetworkX directed multigraph.
- **Predictive Accuracy:** Developed a Random Forest demand forecasting model achieving $R^2 = 0.8942$ and $MAE = 7.42$ passengers using leakage-free temporal features.
- **Anomaly Detection:** Developed a clean-reference Isolation Forest achieving $F1 = 0.9355$, precision = 90.62%, and recall = 96.67% without ground-truth label contamination.
- **Graph Discrepancy Localization:** Implemented a subpath chaining algorithm achieving 95.00% conditional corridor overlap recall for identifying revenue deficit locations.
- **Cloud Streaming Resilience:** Built an asynchronous streaming broker supporting >1,500 transactions/sec with sub-2.5 ms latency, integrated with a 14-endpoint FastAPI backend and database persistence.

8.2 Future Enhancements

While FareGuard demonstrates state-of-the-art accuracy and operational utility, several promising avenues for future research and technical development are identified:

- **Automated Passenger Count (APC) Sensor Fusion** – Integrate optical/infrared sensors to cross-validate passenger boardings with ETM ticket records.
- **Graph Neural Networks (GNNs)** – Extend the subpath localizer with Temporal GNNs to model passenger flows and network congestion.
- **Mobile Auditor Application** – Develop Android/iOS apps for real-time high-risk alerts, bus interception, and navigation.
- **Reinforcement Learning for Patrols** – Use multi-agent reinforcement learning to optimize inspection routes using revenue leakage heatmaps.

REFERENCES

- [1] V. Peluso, M. Di Gangi, and A. Vitetta, "Automated fare collection system data for transit demand modeling: A comprehensive review," *IEEE Transactions on Intelligent Transportation Systems*, vol. 23, no. 8, pp. 10120-10134, Aug. 2022.
- [2] R. Kumar and N. Sharma, "Challenges and vulnerabilities in hybrid electronic ticketing architectures for developing public bus transit networks," in *Proc. IEEE International Conference on Smart Mobility and Transit Systems (ICSMTS)*, Bengaluru, India, 2023, pp. 45-52.
- [3] B. L. Smith and M. J. Demetsky, "Traffic flow forecasting: Comparison of modeling approaches," *Journal of Transportation Engineering*, vol. 123, no. 4, pp. 261-266, 1997.
- [4] G. Moretti, R. C. Alver, and M. G. Santos, "Comparative benchmark of machine learning regressors for short-term urban bus passenger volume forecasting," *Transportation Research Part C: Emerging Technologies*, vol. 119, p. 102758, Oct. 2020.
- [5] Y. Chen, E. K. Lee, and J. Wang, "Gradient boosting and random forest architectures for diurnal passenger load estimation in multi-modal networks," *IEEE Access*, vol. 9, pp. 84210-84224, Jun. 2021.
- [6] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys (CSUR)*, vol. 41, no. 3, pp. 1-58, Jul. 2009.
- [7] F. T. Liu, K. M. Ting, and Z. H. Zhou, "Isolation forest," in *Proc. Eighth IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413-422.
- [8] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, p. 100804, Sep. 2023.
- [9] S. Derrible and C. Kennedy, "The complexity and robustness of metro networks,"
- [10] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. International Conference on Learning Representations (ICLR)*, Toulon, France, 2017.
- [11] M. E. J. Newman, *Networks: An Introduction*. Oxford, UK: Oxford University Press, 2010.
- [12] Bangalore Metropolitan Transport Corporation, "BMTC Open Data Initiative and GTFS Schedule Feeds," Technical Whitepaper, Government of Karnataka, Bengaluru, India, 2024.
- [13] P. Ramirez and H. Gonzalez, "Cloud-native microservices architecture for real-time telemetry processing in smart cities," *IEEE Internet of Things Journal*, vol. 10, no. 4, pp. 3412-3425, Feb. 2023.
- [14] M. Armbrust et al., "A view of cloud computing," *Communications of the ACM*, vol. 53, no. 4, pp. 50-58, Apr. 2010.

APENDIX: CORE SOURCE CODE

https://github.com/EvanKS/FareGuard_Project